

TA-DA! - Improving Activity Recognition Using Temporal Adapters and Data Augmentation

Maximilian Hopp*
University of Siegen
Siegen, Germany
maximilian2.hopp@student.uni-siegen.de

Helge Hartleb*
University of Siegen
Siegen, Germany
helge.hartleb@student.uni-siegen.de

Robin Burchard
University of Siegen
Siegen, Germany
robin.burchard@uni-siegen.de

Abstract

In this report, we describe the technical details of our submission to the WEAR Dataset Challenge 2024. For this competition, we use two approaches to boost the performance of the official WEAR GitHub repository. 1) Integration of a Temporal-Informative adapter (TIA) into the models of the WEAR repository; 2) Data Augmentation Techniques to enrich the provided test dataset. Our method achieves roughly 4.7% improved results¹ on the test set of the WEAR Dataset Challenge 2024 compared to the baseline of the WEAR repository.

CCS Concepts

• **Human-centered computing** → **Ubiquitous and mobile computing**; • **Hardware** → *Sensor applications and deployments*; • **Computing methodologies** → **Activity recognition and understanding**.

Keywords

Human activity recognition; Deep Learning; Machine Learning; Temporal action detection; Data augmentation

ACM Reference Format:

Maximilian Hopp, Helge Hartleb, and Robin Burchard. 2024. TA-DA! - Improving Activity Recognition Using Temporal Adapters and Data Augmentation. In *Companion of the 2024 ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp Companion '24)*, October 5–9, 2024, Melbourne, VIC, Australia. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3675094.3678454>

1 Introduction and Related Work

Activity recognition has become a topic of interest in different fields of computer science. The task of recognizing a certain set of activities from a given data stream remains challenging even with today's standards of machine learning and deep learning models.

*The authors marked with * contributed equally to this work

¹The code to reproduce the experiments is available at: <https://github.com/maxiH137/wear>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

UbiComp Companion '24, October 5–9, 2024, Melbourne, VIC, Australia

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1058-2/24/10

<https://doi.org/10.1145/3675094.3678454>

Especially for non-visual data from a variety of sensors in smart-phones, smartwatches, fitness trackers, and other wearables, the accurate recognition of different activities is still in development. There is a variety of different use cases each consisting of unique datasets and according activities that provide varying degrees of difficulty to predict accurately.

We focus on the detection of human activities from inertial data provided by the WEAR dataset [1]. The data was gathered using four smartwatches, one on each wrist and ankle of the subjects. The subjects took part in two different workouts each consisting of 9 exercises (refer to the original paper for further details on the execution).

The WEAR Challenge 2024 has the goal of training a model for the prediction of 6 previously unseen subjects. The provided GitHub repository features a working temporal action localization (TAL) model utilizing the architectures of Actionformer [9] and Tridet [7]. The use of a TAL model for human activity recognition (HAR) with inertial-based data is a fairly new idea. It was previously used for visual data and adapted for inertial data by serializing the input vectors as pseudo features [2].

Another promising result for TAL in an end-to-end approach is provided by [5]. They utilize a pre-trained TAL model for activity localization on visual data without the need for an explicit feature extractor by using adapters to provide an alternative to fine-tuning with less memory consumption [5].

Recent approaches towards data augmentation [4, 8] for inertial data during training also show substantial improvements in the robustness of HAR models. Normalizing the inputs, adding noise, and permuting the sensor axes are augmentations with possible benefits for reducing overfitting as well as improving generalization to new data.

2 Methodology

In this section we will explain our approach to the WEAR challenge 2024 and outline the changes we made to the existing model.

Our changes are split into two different categories. On the one hand, we changed the architecture of the model's backbone to integrate new layers (see section 2.1) and on the other, we adjusted the training data using different methods of data augmentation (see section 2.2).

2.1 Adapter

TAL models show promising results, even without the use of visual data [2], therefore we decided to look into further improvements

on the already existing model provided in the WEAR GitHub repository [1]. While reviewing recent advancements in the TAL space, we discovered the approach of using end-to-end data on a large scale [5]. For our purposes, the reduction of memory cost is not as significant as for visual datasets. However, the potential shown by the Temporal-Informative adapter (TIA) seemed promising.

The adapter consists of two projections, one reducing the dimensions of the features and one increasing it again, however in between the two projections, a depth-wise convolution as well as a fully connected layer are utilized to improve on the (temporal) receptive field of the adapter. This is further improved upon by an additional skip connection that is added to the output of the fully connected layer [5] (see Fig.1 for the architecture of the TIA).

The TIA was originally intended to be used in between the backbone layers of a pre-trained model (e.g. Actionformer, Tridet, etc. [7, 9]) and improve the generalization of the model to a (new) dataset with lower memory consumption compared to regular fine-tuning, by freezing the other layers during training [5]. Our approach utilizes the TIA in between the backbone layers of Tridet [7] during training to enhance the receptive field of the model therefore improving the ability to find temporal connections in the training data with the hope to make the model more robust on new datasets.

We decided to use the TIA during training since memory usage is not of huge concern when using inertial data. This allowed us to train the adapter simultaneously with the model to improve training efficiency compared to pre-training the model and training the adapter layers afterward.

To implement the TIA into the Tridet model, we used the code provided by the OpenTAD GitHub repository [6] (which has an implementation of the TIA from [5]) and adjusted the (input) dimensions to fit the WEAR dataset. Afterward, we experimented with different architectures of the backbone (see section 3 for more information).

The added adapter layers result in an increase of roughly 7% in trainable parameters compared to the baseline. Therefore, the training time is only marginally increased.

For the training process, we chose to keep the original training parameters from the baseline to achieve a more effective comparison. The parameters of the adapter layers were also kept from [6] as the visual data also had a frame rate of 60Hz which performed best for the baseline. [1]

In addition to using adapters in between the backbone layers, we also adjusted the training data using different augmentations to enhance the model's ability to generalize even better to new data.

2.2 Data Augmentation

The WEAR GitHub repository [1] currently does not implement data augmentation methods in the pre-processing steps of the data. To boost the performance and feed more data to the model, this is a simple extension to enlarge the WEAR Dataset.

As a first step we take the approach described in [4], where a permutation is performed on IMU samples from a smartphone. A single sample consists of the vector $(x_1, y_1, z_1, x_2, y_2, z_2)$, which encodes the acceleration (x_1, y_1, z_1) and orientation (x_2, y_2, z_2) of the

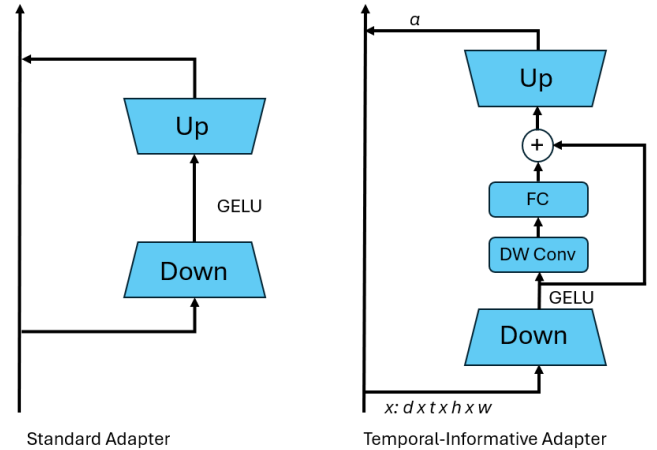


Figure 1: Left: Standard adapter and Right: temporal-informative adapter architectures (adapted from [5]).

device at a certain point in time. The authors reason, that the device that collects the inertial data, can face in any direction in three-dimensional space.

To make the dataset more invariant to this rotation, they permute the (x,y,z) coordinates, delivered by the smartphone.

They restrict their permutation space to mutually orthogonal axes and therefore only consider the following six permutations (x,y,z) , (x,z,y) , (z,y,x) , (z,x,y) , (y,x,z) , (y,z,x) .

Our approach applies the same permutation for the WEAR dataset, where 4 smartwatches collected the IMU samples. Those samples only consist of acceleration data.

Each smartwatch provides a vector (x,y,z) for the current acceleration but not orientation, which means the WEAR dataset delivers twelve values for every sample. One smartwatch was placed on each arm and leg, where the movement of the smartwatch is more restricted than the potential movement of a smartphone. The goal of rotation invariance still makes sense because the WEAR Dataset resembles sports activities where the movement of the smartwatches can not be excluded from the capture process. We follow [4] in creating only six permutations per subject, to avoid unnecessary data.

Our second enhancement is the normalization of the inertial data, for which we again adopt the approach from [4], where each sensor axis was normalized by itself. The authors argued, that by normalizing over all the data, the risk to destroy channel-specific data is high. That is why we also normalize each of the twelve axes separately.

The third and final extension is simply altering the inertial data by downscaling, magnifying, inverting, or adding noise. Our implementation is an adaption of the work and the associated GitHub repository by Wang et al. [8].

Performing those data augmentation increases the amount of training data from 18 subjects to 282 during training. This increased training time by approximately 250% (5h compared to about 1.5h).

Table 1: TriDet score thresholding results ablation experiments on the effect of different score thresholds (0.05, 0.1, 0.15, 0.2 and 0.25) on the raw prediction results (0.0 threshold). We report results with different clip size lengths (CL) (0.5, 1.0 seconds) each with a 50% overlap. We measured the Precision (P), Recall (R), F1 score and mAP scores for thresholds of 0.3 to 0.7. All tests were performed using inertial data only and random seed 1. Our modalities are the Inertial Baseline, Adapter, Data Aug and Adapter + Data Aug. The best results for each modality are in **bold**.

	Threshold	CL	P	R	F1	0.3	0.4	0.5	0.6	0.7	Avg mAP
Inertial Baseline	0.0	0.5s	54.94	77.88	61.53	76.30	73.57	67.90	59.18	49.16	65.22
	0.05	0.5s	68.51	79.26	70.92	75.39	72.72	67.04	58.34	48.35	64.37
	0.1	0.5s	77.28	78.04	76.26	73.34	70.84	65.04	56.14	46.43	62.36
	0.15	0.5s	82.56	75.02	77.19	70.28	67.93	62.10	53.15	43.92	59.48
	0.2	0.5s	86.06	70.10	75.25	66.01	63.71	57.70	49.30	41.09	55.56
	0.25	0.5s	87.78	63.97	71.09	60.73	58.58	52.94	45.06	37.85	51.03
	0.0	1.0s	55.34	78.01	60.87	80.17	78.45	76.11	72.94	67.48	75.03
	0.05	1.0s	66.81	79.22	70.05	79.42	77.66	75.28	72.22	65.73	74.26
	0.1	1.0s	75.83	77.89	75.28	77.92	76.26	73.95	70.86	65.47	72.89
	0.15	1.0s	80.70	75.94	76.91	75.97	74.38	72.16	69.09	64.04	71.13
	0.2	1.0s	83.85	73.76	77.12	73.27	71.66	69.83	66.79	62.25	68.76
	0.25	1.0s	85.73	71.77	76.59	70.96	69.39	67.51	64.72	60.43	66.60
Adapter	0.0	0.5s	57.99	76.47	60.37	74.22	71.93	67.99	59.51	48.5	64.43
	0.05	0.5s	63.11	78.31	71.51	72.82	70.46	66.77	58.37	47.14	63.11
	0.1	0.5s	76.15	75.82	73.76	70.01	67.75	64.22	55.64	45.01	60.53
	0.15	0.5s	80.72	72.65	74.42	66.86	64.86	61.05	52.12	42.01	57.38
	0.2	0.5s	83.92	69.03	73.43	63.48	61.36	57.23	48.78	39.71	54.11
	0.25	0.5s	86.77	63.30	70.28	58.00	56.05	52.93	44.73	36.54	49.65
	0.0	1.0s	56.42	78.21	61.17	80.75	79.17	76.57	72.70	66.59	75.16
	0.05	1.0s	70.01	79.96	72.09	79.81	78.27	75.59	71.77	65.77	74.24
	0.1	1.0s	76.05	79.04	75.64	78.28	76.60	73.90	70.24	64.39	72.68
	0.15	1.0s	80.05	76.75	76.75	75.88	74.06	71.60	68.59	63.00	70.63
	0.2	1.0s	82.82	75.05	77.25	73.80	72.18	69.83	66.97	61.6	68.88
	0.25	1.0s	83.84	73.48	76.91	71.70	70.21	67.81	64.90	59.73	66.87
Data Aug	0.0	0.5s	58.06	79.26	62.34	80.62	78.56	74.79	67.67	59.88	72.30
	0.05	0.5s	72.30	80.98	74.78	79.42	77.27	73.55	66.39	58.53	71.03
	0.1	0.5s	76.50	79.94	76.76	77.71	75.85	72.01	65.24	57.16	69.59
	0.15	0.5s	79.48	78.27	77.42	75.30	73.74	70.10	63.02	54.86	67.41
	0.2	0.5s	82.92	76.55	77.45	72.46	70.98	67.47	60.45	53.03	64.88
	0.25	0.5s	83.66	74.78	76.78	70.33	68.90	65.45	58.41	51.35	62.89
	0.0	1.0s	57.68	80.32	62.58	82.49	81.69	80.13	76.20	72.44	78.59
	0.05	1.0s	74.26	82.38	76.80	80.99	80.26	78.69	74.78	71.14	77.17
	0.1	1.0s	78.15	81.63	78.75	79.43	78.76	77.06	73.39	69.82	75.69
	0.15	1.0s	79.74	80.91	79.31	78.33	77.72	75.97	72.41	68.75	74.64
	0.2	1.0s	81.56	80.11	79.75	77.05	76.53	74.87	71.52	67.63	73.52
	0.25	1.0s	82.29	78.98	79.45	75.75	75.21	73.59	70.77	66.96	72.46
Adapter + Data Aug	0.0	0.5s	54.08	73.93	57.69	70.70	69.34	67.10	62.25	56.73	65.30
	0.05	0.5s	69.91	74.15	70.03	67.37	66.07	64.14	60.07	54.42	62.41
	0.1	0.5s	74.20	70.69	70.36	63.83	62.70	61.31	57.63	52.24	59.54
	0.15	0.5s	75.61	67.46	68.98	60.98	59.87	58.71	55.11	49.99	56.92
	0.2	0.5s	77.66	64.52	67.78	57.95	57.07	55.95	52.56	47.81	54.27
	0.25	0.5s	79.05	61.16	65.50	54.38	53.57	52.58	49.71	45.59	51.17
	0.0	1.0s	57.04	79.11	62.22	80.13	79.46	77.38	74.94	70.50	76.48
	0.05	1.0s	74.09	80.51	75.90	78.49	78.03	75.89	73.37	69.10	74.98
	0.1	1.0s	77.00	79.94	77.30	77.19	76.84	74.78	72.36	67.92	73.82
	0.15	1.0s	78.25	78.73	77.42	75.57	75.27	73.48	71.12	66.69	72.43
	0.2	1.0s	79.50	77.42	77.29	73.37	73.37	72.15	70.02	65.77	71.01
	0.25	1.0s	80.32	76.27	76.99	71.44	70.94	69.78	68.11	64.14	68.88

3 Results

To evaluate the effectiveness of our changes to the backbone of the model as well as the training through our data augmentations, we compare the performance to the baseline values provided in [1]. We split our evaluation into three main categories. Training using only the adapter as part of the backbone, training using only data augmentation and no adapters, and a combined model using both techniques.

We performed evaluation only on the Tridet architecture as its performance was superior to Actionformer for the baseline experiments [1]. Our chosen configurations for the analysis consist of window sizes of 30 frames with a stride of 15 and 60 frames with a stride of 30 (both have 50% overlap of the sliding windows) as those gave the most promising results [1]. The resulting scores can be found in Table 1.

Our results show that we improved the test and validation data performance by roughly 4.7% (from 75.03 avg. mAP to avg. 78.59 mAP) by training on augmented input data.

4 Discussion

As depicted in Table 1, the adapter performs marginally better (75.16 avg. mAP compared to 75.03 avg. mAP) for a threshold of 0.0 (raw prediction results) and a window size of 60 frames (1.0s).

Similar to the baseline, the adapter model performs best with a threshold of 0.0 and a window size of 60 frames. However, the performance is slightly worse for a window size of 30 frames. Most likely this is caused by the adapter's design and parameters that were originally developed for visual data [5] with a 60Hz frame rate (EPIC-KITCHENS dataset [3]).

For the data augmentation, we achieved even better results, improving the mAP from 75.03 to 78.59. As expected, the model using data augmentation also performed best on a window size of 60 frames (1.0s) and a threshold of 0.0 since the baseline model was used without any changes to the backbone or prediction heads. For data augmentation, we used permutation, normalization, scaling, and noise techniques because this combination of data augmentation showed the most promising results.

However, one of the main goals for using data augmentation was to enhance the model's generalization capabilities. Thus, the results will not capture the full picture of the improvement made to the model. To see the advancement in the robustness of predictions, we have to wait for the new subjects that the model will be evaluated on after the end of the challenge.

The combination of both adapters and data augmentations achieved the best results for a 1-second window with 50% overlap. The best average mAP is 76.48, when using a 0.0 score threshold. For the average F1 score a threshold of 0.15 delivered 77.42 as the best result.

Since the combined approach results in a lower average mAP value compared to only using data augmentation, we decided to use the data augmentation model for our submission to the challenge.

One consideration would be that the model could be prone to overfitting with the increase of training data. The data is augmented but the new training data is based on the original 18 subjects and therefore does not have the same quality of data that additional

subjects would provide. However, since the challenge evaluation is score based, our scores are the best prediction on the models performance on new data.

For the combined approach, we are unsure why the use of the TIA performed worse than the baseline model on the augmented dataset and further testing is required.

Potential improvements using the TIA could be achieved by adjusting the training process. Using the TIA for fine-tuning to specific subjects by freezing the backbone after pre-training and only allowing the adapters to adjust their weights could result in a better combined performance since the model performed considerably worse on one of the 18 subjects (subject 9). This also was the intended use of the adapter [5]. It will be part of potential future research.

5 Conclusion

This report shows our solution for the WEAR Dataset Challenge 2024. We tried to improve the baseline results of the WEAR dataset provided by the WEAR repository [1] by adapting the adapter approach from [5] and adding data augmentation techniques like permutation, normalization, and data altering techniques from [4, 8]. The results are promising for the prediction of unseen data, however, more research into the combination of the adapter and data augmentation approaches is still necessary.

6 Acknowledgments

This work was supported by the University of Siegen. Special thanks to Robin Burchard for his support with the submission process.

References

- [1] Marius Bock, Hilde Kuehne, Kristof Van Laerhoven, and Michael Moeller. 2023. WEAR: An Outdoor Sports Dataset for Wearable and Egocentric Activity Recognition. (11 2023). <https://doi.org/10.48550/arXiv.2304.05088>
- [2] Marius Bock, Michael Moeller, and Kristof Van Laerhoven. 2023. Temporal Action Localization for Inertial-based Human Activity Recognition. *arXiv preprint arXiv:2311.15831* (2023).
- [3] Dima Damen, Hazel Doughty, Giovanni Maria Farinella, Sanja Fidler, Antonino Furnari, Evangelos Kazakos, Davide Moltisanti, Jonathan Munro, Toby Perrett, Will Price, and Michael Wray. 2018. Scaling Egocentric Vision: The EPIC-KITCHENS Dataset. In *European Conference on Computer Vision (ECCV)*.
- [4] Zhiqing Hong, Zelong Li, Shuxin Zhong, Wenjun Lyu, Haotian Wang, Yi Ding, Tian He, and Zhang Desheng. 2024. CrossHAR: Generalizing Cross-dataset Human Activity Recognition via Hierarchical Self-Supervised Pretraining. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 8 (05 2024), 1–26. <https://doi.org/10.1145/3659597>
- [5] Shuming Liu, Chen-Lin Zhang, Chen Zhao, and Bernard Ghanem. 2024. End-to-End Temporal Action Detection with 1B Parameters Across 1000 Frames. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 18591–18601.
- [6] Shuming Liu, Chen Zhao, Fatimah Zohra, Mattia Soldan, Carlos Hinojosa, Alejandro Pardo, Anthony Cioppa, Lama Alssum, Mengmeng Xu, Meray Ramazanova, Juan León Alcázar, Silvio Giancola, and Bernard Ghanem. 2024. OpenTAD: An Open-Source Toolbox for Temporal Action Detection. <https://github.com/sming256/opentad>.
- [7] Dingfeng Shi, Yujie Zhong, Qiong Cao, Lin Ma, Jia Li, and Dacheng Tao. 2023. Tridet: Temporal action detection with relative boundary modeling. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 18857–18866.
- [8] Jinqiang Wang, Tao Zhu, Jingyuan Gan, Liming Luke Chen, Huansheng Ning, and Yaping Wan. 2022. Sensor Data Augmentation by Resampling in Contrastive Learning for Human Activity Recognition. *IEEE Sensors Journal* 22, 23 (2022), 22994–23008. <https://doi.org/10.1109/JSEN.2022.3214198>
- [9] Chen-Lin Zhang, Jianxin Wu, and Yin Li. 2022. Actionformer: Localizing moments of actions with transformers. In *European Conference on Computer Vision*. Springer, 492–510.